

Documentation technique

Data Lake financier

Groupe : Artemiy Smogunov, Nathan Smadja-Tubiana, Patrice Ignongui

Projet EFREI 2025-2026

Data Lake financier

Projet final du cours Data Lakes & Data Integration, EFREI 2025-2026.

Auteurs :

- Artemiy Smogunov
- Nathan Smadja-Tubiana
- Patrice Ignongui

Le sujet est conservé dans consignes/Data_Lakes_Projet_Final_EFREI_2025-2026.pdf.

Ce que fait le projet

Le projet collecte des cours financiers journaliers depuis deux sources :

- data/finance_dataset.csv pour la source fichier ;
- Yahoo Finance avec yfinance pour la source API.

Les données passent dans trois zones.

```
CSV -----|
              |--> Raw --> Staging --> Curated --> FastAPI
Yahoo Finance -|              Airflow orchestre le flux
```

Raw conserve les objets dans MinIO et les documents dans Elasticsearch. Staging nettoie les données et calcule les indicateurs financiers dans PostgreSQL. Curated ajoute la détection d'anomalies, la tendance et le signal. FastAPI donne accès aux trois zones.

Organisation du dépôt

api/	application FastAPI et routes HTTP
airflow/dags/	définition du DAG
src/financial_data_lake/	package Python réutilisable
config/	configuration commune
ingestion/	ingestion fichier et Yahoo Finance
transformation/staging/	nettoyage et indicateurs
transformation/curated/	anomalies, tendances et signaux
data/	fichier CSV versionné
scripts/	initialisation, benchmark et génération PDF
tests/	tests Python
docs/captures/	captures de l'exécution vérifiée
livrables/	rapport Markdown et PDF

Le code métier est dans src/financial_data_lake. L'API et le DAG l'importent comme un package Python. Cette structure évite de recopier les fonctions dans plusieurs scripts et permet aussi de les importer depuis un autre projet après installation du package.

Environnement Python avec uv

Les dépendances se trouvent dans `pyproject.toml`. Le fichier `uv.lock` conserve les versions exactes résolues.

Prérequis : Python 3.11 ou 3.12 et uv.

```
git clone https://github.com/Nathan2412/Projet-DataLake-Final.git
cd Projet-DataLake-Final
uv sync --frozen
uv run pytest -q
```

`uv sync --frozen` crée `.venv`, installe le package `financial-data-lake` depuis `src` et installe les dépendances de développement. Lors de notre vérification, uv a utilisé Python 3.11.15. Les 13 tests ont réussi.

Lancement avec Docker Compose

Prérequis : Docker et Docker Compose.

```
docker compose up -d --build
```

Le premier démarrage crée les tables PostgreSQL, les buckets MinIO et l'utilisateur Airflow.

Contrôle des conteneurs et de l'API :

```
docker compose ps
curl http://localhost:8000/health
```

Interfaces locales :

- FastAPI : `http://localhost:8000/docs`
- Airflow : `http://localhost:8080`
- MinIO : `http://localhost:9001`

Les identifiants utilisés pour cet environnement local sont dans `docker-compose.yml`.

Services Docker

PostgreSQL écoute sur le port 5432. Il contient les tables Staging, Curated, les journaux d'ingestion et la base interne d'Airflow.

MinIO écoute sur les ports 9000 et 9001. Les buckets `raw-financial-data` et `raw-api-data` conservent les objets Raw.

Elasticsearch 8.11.0 écoute sur le port 9200. L'index `raw_financial_events` contient les lignes Raw utilisées par l'API et la transformation Staging.

Airflow 2.8.1 écoute sur le port 8080. Le scheduler exécute le DAG et le webserver affiche les exécutions.

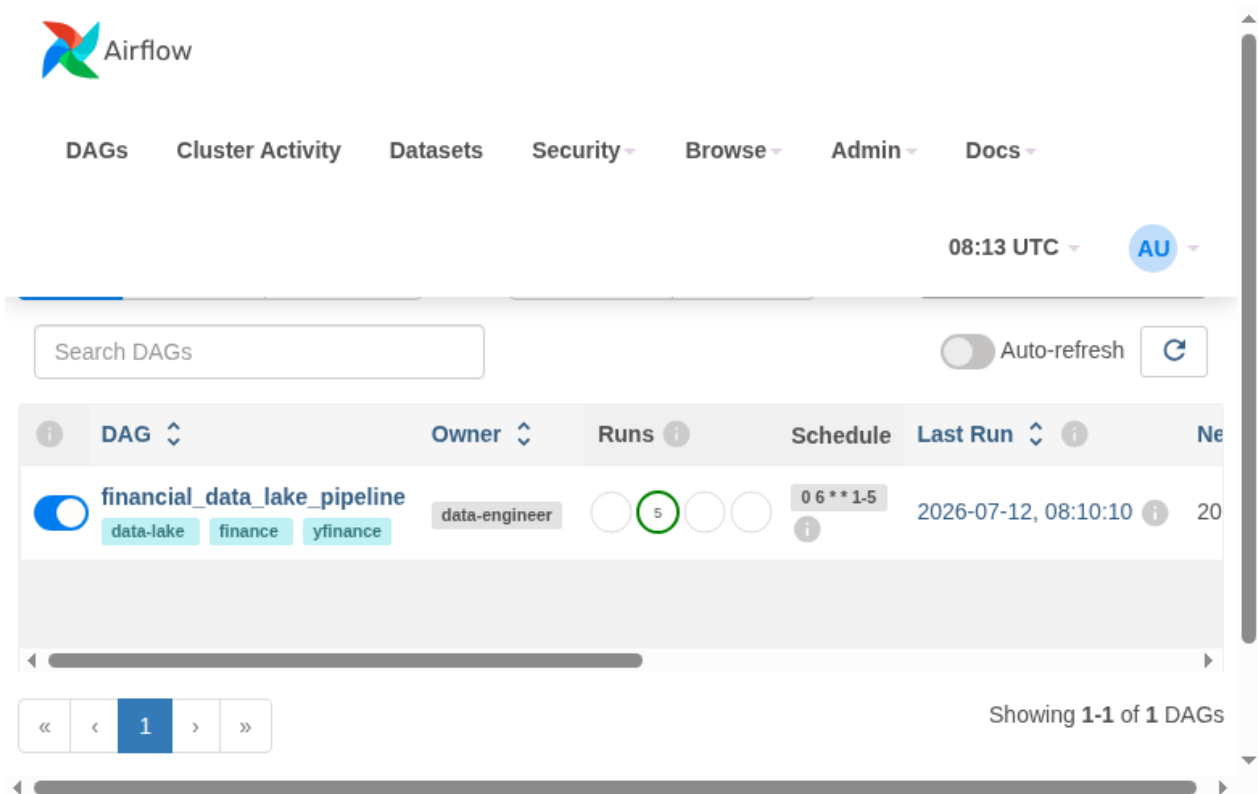
FastAPI écoute sur le port 8000. L'image copie src/ dans /app/src et utilise ce chemin dans PYTHONPATH.

Pipeline Airflow

Le DAG s'appelle `financial_data_lake_pipeline`. Il est planifié à 6 h UTC du lundi au vendredi.

```
start
|-- ingest_file --|
|                  |--> transform_staging --> transform_curated --> log_summary --> end
|-- ingest_api  --|
```

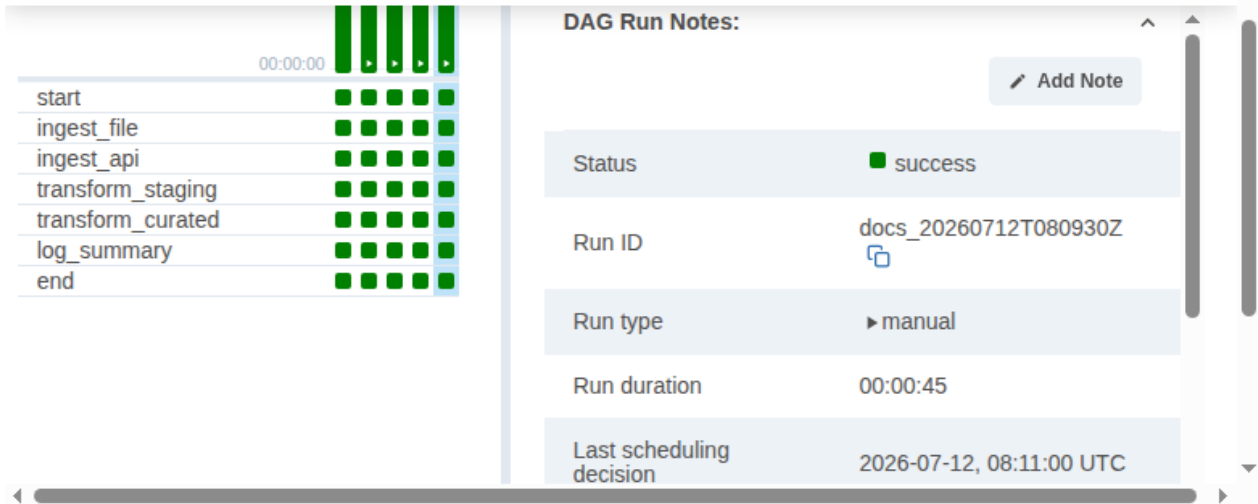
Les deux ingestions commencent en parallèle. Elles placent les tickers réussis dans XCom. Staging traite l'union de ces tickers. Curated traite ensuite les tickers réussis dans Staging.



The screenshot shows the Airflow web interface. At the top, there's a navigation bar with links: DAGs, Cluster Activity, Datasets, Security, Browse, Admin, and Docs. The current time is 08:13 UTC. Below the navigation bar, there's a search bar for DAGs and an 'Auto-refresh' toggle. The main content area displays a table of DAGs. The first DAG is 'financial_data_lake_pipeline', owned by 'data-engineer'. It has a status of 'Running' (indicated by a green circle with a '5' inside). The schedule is '0 6 * * 1-5'. The last run was on '2026-07-12, 08:10:10'. The DAG is associated with tags 'data-lake', 'finance', and 'yfinance'. At the bottom, there's a pagination bar showing 'Showing 1-1 of 1 DAGs'.

Liste des DAGs dans Airflow

Cette capture a été prise après la reconstruction de la stack. Elle montre le DAG chargé par Airflow. La commande `airflow dags list-import-errors` n'a retourné aucune erreur.



Exécution du DAG

Cette capture correspond au run docs_20260712T080930Z. Le run a commencé à 08:10:15 UTC et s'est terminé à 08:11:00 UTC. Les sept tâches ont le statut success.

Pourquoi les imports du DAG sont dans les tâches

Airflow lit souvent le fichier du DAG pour afficher son graphe. Les imports des modules métier restent donc dans les fonctions `task_*`. Les bibliothèques d'ingestion et de transformation sont chargées lorsque la tâche démarre, pas à chaque lecture du fichier. Un commentaire simple dans `airflow/dags/financial_pipeline_dag.py` explique ce choix.

Zone Raw

L'ingestion fichier lit `data/finance_dataset.csv`. Elle vérifie les colonnes obligatoires, convertit les dates et les nombres, enlève les lignes incomplètes et garde la dernière ligne pour un doublon (`ticker, date`).

L'ingestion API télécharge les cours avec `yfinance`. Elle conserve un JSON complet dans MinIO puis indexe les lignes dans Elasticsearch.

L'identifiant Elasticsearch suit la forme `source_ticker_date`. Une nouvelle ingestion de la même source, du même ticker et de la même date met à jour le document existant.

Zone Staging

Staging relit les documents Elasticsearch par ticker. Le code trie les dates, enlève les doublons et ignore les lignes sans cours de clôture.

Les indicateurs calculés sont :

- SMA 20 et SMA 50 ;
- EMA 12 et EMA 26 ;
- RSI 14 ;
- MACD et ligne de signal ;
- bandes de Bollinger ;
- rendement journalier ;
- volatilité sur 20 périodes.

PostgreSQL utilise la clé unique (ticker, date). Une nouvelle exécution met la ligne à jour au lieu de la dupliquer.

Zone Curated

Curated lit les lignes Staging par ticker. Isolation Forest utilise le rendement journalier, la volatilité, le z-score du volume et le RSI. Le modèle utilise 100 arbres, random_state=42 et une contamination de 5 %. Il ne démarre pas sous 30 lignes pour un ticker.

Les types d'anomalies sont attribués avec les règles du fichier
src/financial_data_lake/transformation/curated/transform_curated.py :

- rendement sous -5 % : flash_crash ;
- rendement au-dessus de 5 % : price_spike ;
- z-score absolu du volume au-dessus de 3 : volume_spike ;
- volatilité au-dessus de 4 % : high_volatility ;
- autre point isolé : unknown_anomaly.

La tendance compare le cours, la SMA 20 et la SMA 50. Le signal combine le RSI et le MACD. Le signal vaut buy, sell ou hold. Il s'agit d'une sortie technique du projet, pas d'un conseil financier.

API

Financial Data Lake API 1.0.0 OAS 3.1

[OpenAPI JSON](#)

API Gateway du data lake financier (Yahoo Finance / yfinance). Expose les trois zones du data lake (Raw, Staging, Curated) et fournit deux endpoints d'ingestion.

Health	^
GET /health	Verification de l'état des services
Stats	^
GET /stats	Métriques de remplissage du data lake
Raw	^
GET /raw	Données brutes de la zone Raw
GET /raw/objects	Liste des objets MinIO dans la zone Raw
Staging	^
GET /staging	Données staging avec indicateurs techniques
GET /staging/tickers	Liste des tickers disponibles en staging
Curated	^
GET /curated	Données curated avec scores d'anomalie
GET /curated/anomalies/summary	Résumé des anomalies par ticker
GET /curated/signals	Tickers avec signaux actifs (buy/sell)
Ingest	^
POST /ingest	Ingestion synchrone avec benchmark
Ingest Fast	^
POST /ingest_fast	Ingestion parallèle
Schemas	^
HTTPValidationError	Expand all object
HealthResponse	Expand all object
IngestFastRequest	Expand all object
IngestFastResponse	Expand all object
IngestRequest	Expand all object
IngestResponse	Expand all object
ServiceStatus	Expand all object
ValidationError	Expand all object

Routes Swagger

La capture montre les routes réellement enregistrées dans FastAPI :

- GET /health contrôle PostgreSQL, MinIO et Elasticsearch ;
- GET /stats retourne les volumes des trois zones ;
- GET /raw et GET /raw/objects lisent la zone Raw ;
- GET /staging et GET /staging/tickers lisent Staging ;
- GET /curated, /curated/anomalies/summary et /curated/signals lisent Curated ;
- POST /ingest exécute le pipeline séquentiel ;
- POST /ingest_fast utilise des téléchargements et des écritures groupés.

Exemples :

```
curl http://localhost:8000/health
curl http://localhost:8000/stats
curl "http://localhost:8000/raw?ticker=AAPL&limit=1"
curl "http://localhost:8000/staging?ticker=AAPL&limit=1"
```

```
curl "http://localhost:8000/curated?ticker=AAPL&limit=1"
```

Exécution vérifiée après la migration vers src

Le 12 juillet 2026, nous avons reconstruit les images et les conteneurs avec `docker compose up -d --build --force-recreate`. Nous avons ensuite déclenché le run Airflow `docs_20260712T080930Z`.

Exécution contrôlée après la migration vers src

Run Airflow : `docs_20260712T080930Z`

État du DAG	success
Début	08:10:15 UTC
Fin	08:11:00 UTC
Tâches réussies	7 / 7
PostgreSQL	ok
MinIO	ok
Elasticsearch 8.11.0	ok
Objets MinIO	271
Documents Elasticsearch	1 777
Lignes Staging	1 254
Lignes Curated	1 254
Anomalies	37

Valeurs relevées avec `GET /health`, `GET /stats` et les états de tâches Airflow.

Résultats de l'exécution

Résultats relevés après le run :

- API : ok ;
- PostgreSQL : ok ;
- MinIO : ok ;
- Elasticsearch 8.11.0 : ok ;
- 271 objets MinIO, dont 201 objets fichier et 70 objets API ;
- 1 777 documents Elasticsearch ;
- 1 254 lignes Staging ;
- 1 254 lignes Curated ;
- 37 anomalies ;

- sept tâches Airflow réussies.

MinIO contient davantage d'objets après chaque exécution car les objets API sont horodatés. Elasticsearch utilise des identifiants stables et ne duplique pas une même combinaison source, ticker et date. Staging et Curated ont le même nombre de lignes car Curated enrichit chaque ligne Staging.

Exemple AAPL dans les trois zones

AAPL dans Raw, Staging et Curated

Dernière date retournée par les trois routes : 10 juillet 2026.

Raw — Elasticsearch

```
ticker: AAPL      close: 314.539886
volume: 14 262 307 source: yfinance_fast
date: 2026-07-10
```

Staging — PostgreSQL

```
ticker: AAPL      close: 315.320007
sma_20: 254.378001 sma_50: 239.001591
rsi_14: 89.5685   macd: 22.881679
volatility_20: 0.066898
```

Curated — PostgreSQL

```
ticker: AAPL      close: 315.320007
is_anomaly: true  anomaly_type: high_volatility
price_trend: bullish
signal: hold      anomaly_score: -0.700893
```

Relevé après le run docs_20260712T080930Z.

AAPL dans Raw, Staging et Curated

Les trois routes ont retourné une ligne AAPL datée du 10 juillet 2026.

Raw contient le cours reçu, le volume, la source et l'heure d'ingestion. Staging contient les indicateurs recalculés depuis l'historique disponible. Curated reprend la ligne Staging et ajoute l'anomalie, la tendance et le signal.

Dans le résultat observé, Curated classe la ligne comme `high_volatility`. La tendance vaut `bullish` et le signal reste `hold`. Les valeurs Raw et Staging ne sont pas strictement identiques car plusieurs sources peuvent exister pour la même date et Staging déduplique par (`ticker`, `date`) après lecture de l'index.

Tests et vérifications

```
uv run pytest -q
sudo docker compose config -q
sudo docker compose exec -T airflow-scheduler airflow dags list-import-errors
```

Résultat final : 13 tests réussis et aucune erreur d'import Airflow.

Les tests couvrent les indicateurs, les tendances, les signaux, la préparation Staging, les identifiants Raw, le chargement du CSV, le benchmark et la présence du package sous src.

Limites observées

Yahoo Finance est externe au projet. Une requête peut être lente ou ne retourner aucune ligne.

Le RSI demande quatorze observations. Un petit lot de cinq jours ne suffit pas pour calculer cet indicateur seul.

Isolation Forest ne s'exécute pas sous 30 lignes pour un ticker. La contamination de 5 % règle la proportion attendue de points isolés ; elle ne prouve pas qu'un mouvement de marché est une erreur.

Les signaux n'ont pas été évalués comme stratégie d'investissement.

Les mots de passe du fichier Compose servent uniquement à l'environnement local du cours.

Livrables

- livrables/RAPPORT_TECHNIQUE.md
- livrables/Documentation_Technique_DataLake_Finance_Artemiy_Smogunov_Nathan_Smadja-Tubiana_Patrice_Ignongui.pdf
- livrables/Rapport_DataLake_Finance_Artemiy_Smogunov_Nathan_Smadja-Tubiana_Patrice_Ignongui.pdf
- docs/CONFORMITE_DEVOIR.md

Régénération des PDF :

```
uv run python scripts/generate_pdf_deliverables.py
```

Vérification par rapport au sujet

Sujet : Data Lakes & Data Integration, projet final EFREI 2025-2026.

Cette page relie les demandes du sujet aux fichiers et aux résultats observés le 10 juillet 2026.

Data lake de bout en bout

Le flux exécuté est le suivant :

Deux sources --> Raw --> Staging --> Curated --> API

Preuves dans le dépôt :

- ingestion : `src/financial_data_lake/ingestion/ingest_file.py` et `src/financial_data_lake/ingestion/ingest_api.py` ;
- transformation :
`src/financial_data_lake/transformation/staging/transform_staging.py` et `src/financial_data_lake/transformation/curated/transform_curated.py` ;
- stockage : `docker-compose.yml` et `scripts/init_db.sql` ;
- orchestration : `airflow/dags/financial_pipeline_dag.py` ;
- exposition : `api/main.py` et `api/routers/`.

Deux sources

Le projet utilise réellement :

- `data/finance_dataset.csv` comme source fichier versionnée ;
- Yahoo Finance par la bibliothèque `yfinance` comme source API.

L'exécution contrôlée a produit 200 objets dans le bucket fichier et 42 objets dans le bucket API.

Trois zones

Raw utilise deux stockages :

- MinIO pour les objets CSV et JSON ;
- Elasticsearch pour les lignes brutes interrogeables.

Staging utilise la table PostgreSQL `staging_ohlcv`.

Curated utilise la table PostgreSQL `curated_analysis`.

Après l'exécution, Staging et Curated contenaient chacun 1 254 lignes.

Pipeline Airflow

Le DAG `financial_data_lake_pipeline` exécute sept tâches. Le run `docs_20260712T080930Z` s'est terminé en 45 secondes avec sept statuts success.

La capture se trouve dans `docs/captures/airflow-execution.png`.

Le planning du code est `0 6 * * 1-5`, soit 6 h UTC du lundi au vendredi.

API Gateway

FastAPI expose les routes demandées :

- `/raw` ;
- `/staging` ;
- `/curated` ;
- `/health` ;
- `/stats`.

Le projet ajoute aussi `/raw/objects`, `/staging/tickers`, `/curated/anomalies/summary`, `/curated/signals`, `/ingest` et `/ingest_fast`.

La capture Swagger se trouve dans `docs/captures/api-swagger.png`.

Traitement avancé

La zone Curated applique Isolation Forest à quatre variables et enregistre le score, le booléen d'anomalie et son type. Elle calcule aussi une tendance et un signal.

L'état observé contenait 37 anomalies sur 1 254 lignes Curated.

Ingestion optimisée

`/ingest` traite les téléchargements séquentiellement.

`/ingest_fast` utilise :

- huit threads pour les téléchargements ;
- huit threads pour les envois MinIO ;
- une écriture Elasticsearch groupée ;
- `execute_values` pour l'upsert PostgreSQL.

Le benchmark enregistré compare les deux routes avec trois répétitions.

Pour 100 tickers, la médiane passe de 104 825,65 ms à 12 230,36 ms. Le gain mesuré est 88,33 %. Les mesures et statuts de chaque appel sont dans `livrables/benchmark_ingest_vs_ingest_fast.json`.

Documentation et captures

Le dépôt contient :

- le guide de lancement dans README.md ;
- le rapport dans livrables/RAPPORT_TECHNIQUE.md ;
- cette vérification ;
- la description du CSV dans data/README.md ;
- cinq captures dans docs/captures/ ;
- deux PDF produits par scripts/generate_pdf_deliverables.py.

Contrôles exécutés

```
docker compose up -d
curl http://localhost:8000/health
curl http://localhost:8000/stats
docker compose exec airflow-scheduler airflow dags trigger financial_data_lake_pipeline
PYTHONPATH=./api .venv/bin/python -m pytest -q
```

Résultats :

- services PostgreSQL, MinIO et Elasticsearch en état ok ;
- run Airflow terminé avec sept tâches en succès ;
- endpoints Raw, Staging et Curated accessibles ;
- 12 tests réussis.